English-to-Khmer Phonetic Transliteration: Learning from Noisy Web Pairs and Measuring Acceptability

Richard Cheam
richardcheam912@gmail.com

Abstract

A Khmer speech system that meets an English word such as *taxi* must first write it in Khmer script, in a way a Khmer reader would pronounce like the English word. We found no suitable curated training dataset or established benchmark for this task, so we trained models on English–Khmer word pairs found in web text and asked how to tell whether their spellings are any good. We built a small character-level model from scratch and adapted a pretrained byte-level model, and we released both. The central lesson concerns measurement. Scoring a spelling by exact agreement with one reference spelling proved misleading: it rejected spellings a Khmer reader accepted and rewarded references the reader rejected. Blinded human review, in which spellings are judged acceptable, not acceptable or uncertain, changed which conclusions we could draw. Under that review, cleaner and larger real data looked encouraging, whereas longer training and adding English pronunciation information gave no clear improvement. Irregularly spelled English words remain the main weakness, and acceptable spellings are often present among a model's alternatives without being chosen first.

1 Problem and motivation

Khmer text often contains English words and names. A Khmer text-to-speech system reads only Khmer script, so a front end must rewrite each English word in Khmer letters before the speech model sees it. The spelling has to make a Khmer reader say something close to the *English* pronunciation: *london* becomes ឡុនដុន, read roughly as "lon-don".

This task is not translation; the meaning of the word plays no part. Nor is it the conversion of *romanised Khmer* (Khmer words typed in Latin letters, such as *sok sabay* for សុខសប្បាយ) back into Khmer script. Here the input is a genuine English word and the target is a Khmer approximation of its English sound.

The target also differs from established loan spellings. Khmer writers usually spell *coffee* កាហ្វេ, a form that does not follow the English vowel sounds. A pronunciation-based spelling such as ខូហ្វី is closer to English. Both appear in our web data, and only the second follows the task's target.

More than one spelling can be right. Khmer has two series of consonant letters, inherent vowels, stacked consonants and small marks that shift a consonant's series, and many English sounds have no exact equivalent. English *f*, for example, can reasonably be written ហ្វ or ផ. A useful system therefore needs to produce *an* acceptable spelling, not *the* spelling.

The central engineering difficulty is supervision. We found no curated English–Khmer phonetic dictionary to learn from and no established test set that lists the acceptable spellings of each word. We had to build training data from noisy web text and, just as importantly, find a way of measuring quality that a Khmer reader would agree with.

2 Building a transliterator with limited data

2.1 Two kinds of model

We tried two approaches with different costs (Table 1).

The first is a *small character-level model*: a standard encoder–decoder Transformer [16] of 2.6 million parameters, trained from scratch to map English letters to Khmer letters. It runs in tens of milliseconds on a CPU with nothing but NumPy.

The second adapts *ByT5-small* [18], a 300-million-parameter text-to-text model that reads and writes raw UTF-8 bytes instead of a fixed vocabulary of word pieces. Working on bytes suits a task about spelling, and ByT5 has been used for transliteration of romanised South Asian text [8]. We adapted it with *LoRA* [6], which keeps the pretrained weights frozen and trains a small set of additional low-rank matrices (here about six million parameters) in the attention and feed-forward layers.

Figure 1 shows how each model turns the same English word into Khmer script. Both are encoder–decoder models: the encoder reads the whole input, and the decoder writes the output one unit at a time, feeding each new unit back in to predict the next. They differ in their units and in what is trained.

2.2 What the models learn

Let x be the token sequence of the English input and $y = (y_1, \ldots, y_T)$ the token sequence of its Khmer spelling, whose last token y_T is the end symbol (the *end* boxes in Figure 1). The tokens differ between the models. For the small model, x is the task marker followed by one token per letter, and each y_t is one

Khmer Unicode character. For ByT5, x is the word's UTF-8 bytes followed by an end symbol, and each y_t is one byte. Both models factorise the probability of a spelling autoregressively [14]:

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x), \tag{1}$$

where θ denotes the trained weights and $y_{<t} = (y_1, \ldots, y_{t-1})$ is the output prefix, which is empty at $t = 1$, when the decoder sees only the start symbol. In words, each output token is predicted from the whole input (through the encoder states) and from the tokens already written; this is the feedback loop drawn as dashed arrows in Figure 1. During training the decoder receives the reference prefix; at inference it receives the prefix it has generated itself. Each Khmer character costs one token in the small model but three in ByT5 (in Figure 1, 7 characters versus 18 bytes, each followed by the end symbol), so sequence lengths and per-token losses of the two models are not directly comparable.

Training minimises the negative log-likelihood of the recorded spelling,

$$\mathcal{L}(x, y) = -\sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x). \tag{2}$$

In both models the padding that makes a batch rectangular is excluded, and the loss is averaged over all real target tokens in a batch, so a longer spelling contributes more terms. The small model replaces the one-hot target in Equation (2) with label smoothing of 0.1 [15] (Equation (5) in Appendix A); the adapted model uses Equation (2) unchanged. A word with several mined spellings appears as several training pairs, each with its own loss: training raises the probability of every recorded spelling, but it does not sum probability over a set of accepted spellings.

For ByT5, LoRA [6] keeps each pretrained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ of an adapted projection frozen and learns a low-rank correction:

$$W_{\text{adapted}} = W_0 + \frac{\alpha}{r} BA, \tag{3}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are learned, the rank is $r = 16$ and the scale is $\alpha = 32$, so $\alpha/r = 2$. B starts at zero, so training begins from the unchanged pretrained model. Only A and B, the orange *LoRA adapter* blocks in Figure 1B, are learned: θ is every weight of the small model but only the adapter matrices of ByT5. The adapted projections and their sizes are listed in Appendix A.

At inference the ideal answer is the most probable spelling, which can only be searched for approximately:

$$\hat{y} \approx \arg\max_{y} \sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x). \tag{4}$$

Greedy decoding takes the most probable token at each step. Beam search of width 4 keeps the four prefixes with the highest summed log-probability, with no length normalisation, and completes each candidate at the end symbol. The small model answers with its top beam candidate and the adapted model with its greedy answer.

Table 1: The two released models. Training data are paired English words and Khmer spellings.

	Small model	Adapted ByT5
Approach	Transformer trained from scratch	pretrained byte model + LoRA adapter
Size	2.6M parameters	300M frozen + 6M trained
Training pairs	about 9,700	about 4,800 (3,585 words)
Data	mined web pairs	mined, recovered and corpus-attested pairs
Decoding	beam search (width 4)	greedy; beam search for candidates

Equation (2) rewards reproducing one recorded spelling, whereas the human evaluation of Section 3 accepts any spelling a Khmer reader would pronounce close to the English word, and may accept several for one word. A lower training loss need not mean more acceptable spellings.

Byte-level adaptation has a pitfall that an engineer can easily mistake for a bug. Every Khmer character takes three UTF-8 bytes, and two of them are almost free to predict. Our first adaptation, which adapted only the attention layers for a short time, produced the same spelling (ហ្វេន) for 204 of 228 evaluation words while its per-byte training loss (Equation (2)) fell steadily: the model had learned to write plausible Khmer bytes without reading the input. An audit of the training and decoding pipeline found no defect that explained the collapse. Adapting the feed-forward layers as well and training longer produced the first working adapter, one that follows its input; we did not isolate which of the two changes mattered.

2.3 Where the paired examples come from

Khmer news and encyclopaedia text sometimes writes a foreign word in Khmer script followed by the original in brackets. Searching the Khmer part of the FineWeb-2 web corpus [12] and Khmer Wikipedia for this pattern yields paired examples such as ហ្វេសប៊ុក (Facebook), shown in context in Figure 2. We kept a pair only if the English word has an entry in the CMU Pronouncing Dictionary [1], so that its intended pronunciation is known, and passed it through the checks described below. A further set of names and loanwords was taken from human translations in the ALT Parallel Corpus [13], where the Khmer spelling occurs verbatim. None of these spellings was reviewed by a person before training.

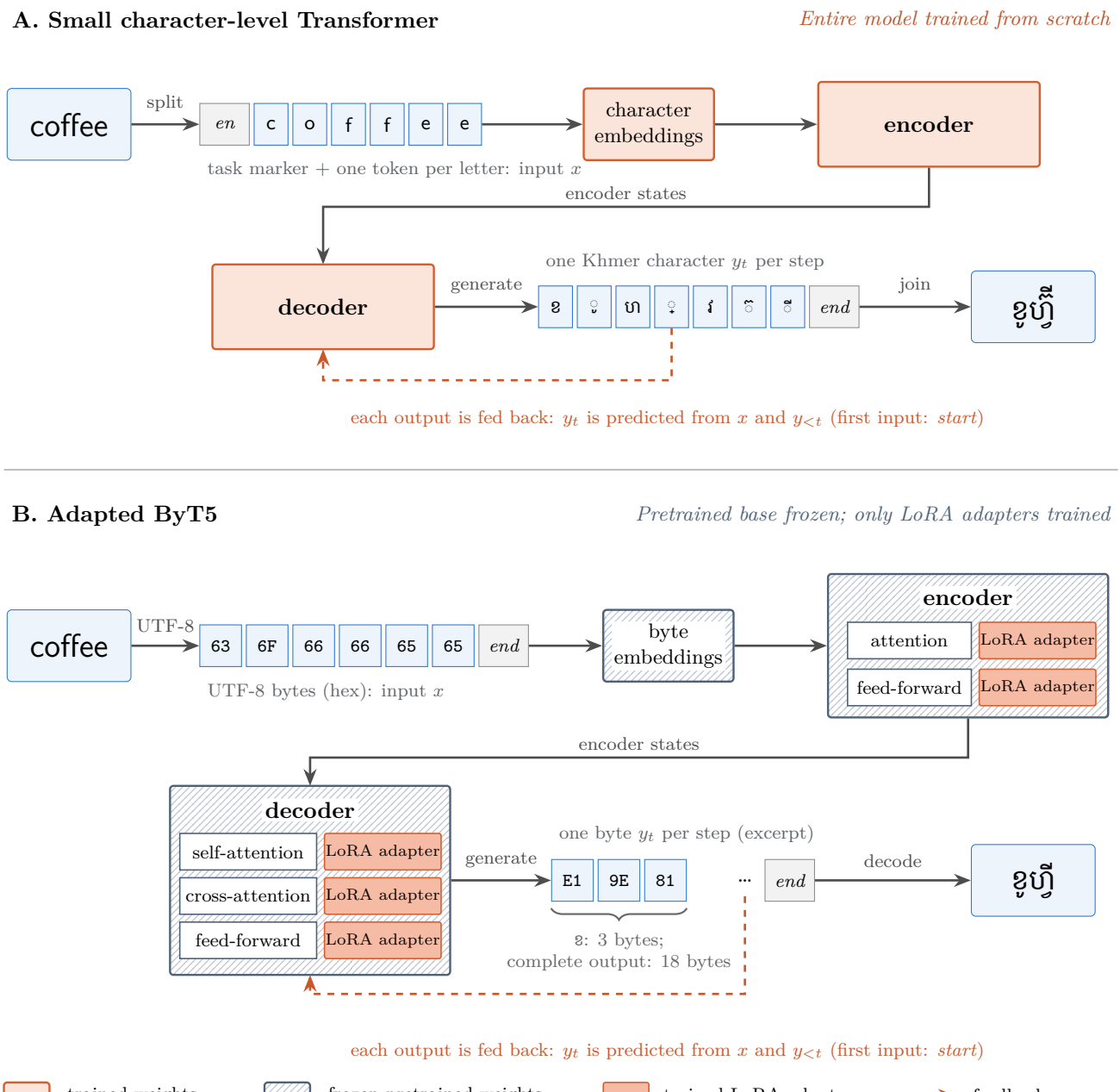


Figure 1: How the two released models turn an English word into Khmer script; both read the same English spelling and write Khmer text. (A) The small model reads a task marker and one token per letter; its encoder (3 layers) passes its states to the decoder (3 layers), which writes one Khmer character at a time. All of its weights are trained from scratch. (B) The adapted ByT5 reads the UTF-8 bytes of the word (the model's token for each byte is the byte value plus 3) and writes Khmer as bytes, three per Khmer character, which are then decoded into text. Its pretrained encoder (12 layers) and decoder (4 layers) stay frozen; only the small LoRA adapters added beside every attention and feed-forward block are trained. In both models, generation starts from a start symbol, each output is fed back to predict the next (dashed arrows), and generation stops when the model emits an end symbol. The outputs are the recorded spellings of *coffee* by each released model. Notation as in Section 2.2: x is the input sequence, y_t the output token at step t, and the final *end* token is y_T.

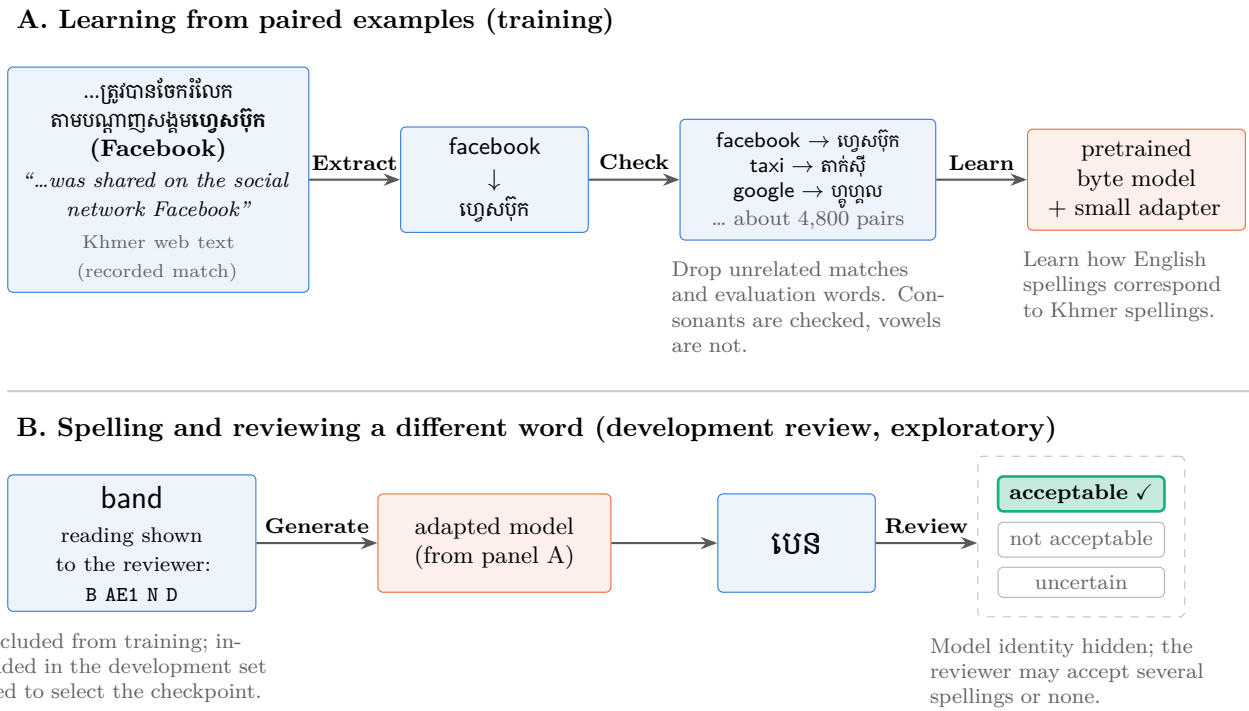


Figure 2: How the adapted model is trained and evaluated. (A) A Khmer spelling followed by its English original in brackets is extracted as a training pair; after checks, thousands of such pairs teach the model the correspondence. (B) A development word, excluded from training but part of the set used to select the checkpoint, is spelled by the model, and a Khmer reader judges the result against the English reading. This review is exploratory; the main results come from two separate prospective tests of fresh words. Both panels use recorded data: the Facebook match and training pairs come from the mining records, and *band*, its output and the judgment from the development review.

A filtering mistake that changed what the model could learn. The miner was also meant to discard romanised Khmer that happens to sit in brackets. Its rule dropped a pair if the Latin side was a known romanised Khmer word *or* if the Khmer side was a known Khmer word. The second condition quietly removed every pair whose Khmer spelling appears in a Khmer dictionary, which includes well-known loans such as ហ្វេសប៊ុក. As a result, none of the small model's training targets could be found in the dictionary, a sign that revealed the error once noticed. Splitting the two conditions returned 11,377 candidate pairs to the pool. The general lesson is to inspect what a filter actually removes, not only how much.

What automatic checks can and cannot verify. To discard unrelated matches, a rule-based reader converts the Khmer consonants into sound classes and compares them with the consonants of the English pronunciation. On held-out dictionary words it agreed with the dictionary's own transcriptions for about 88.6% of words, so it is a useful consonant check. It does not read vowels at all, and vowel differences turned out to be the largest single kind of disagreement between model outputs and references (41–49% of mismatches). Some valid pairs failed the consonant check because of doubled letters or glides; a later pass recovered 1,292 of them, which also admits some noise (the recovered pairs include a truncated *hotel* → តែល).

Keeping evaluation words separate. English words come in families such as *share*, *shares*, *sharing* and *shared*, and a model that has seen *shared* has nearly seen *share*. We therefore grouped words into families with a simple suffix-stripping rule (removing endings such as *-s*, *-ed* and *-ing*, and joining words that share a stem) and kept every family on one side of each split. This is a heuristic, not a full morphological analysis: irregular forms such as *ran* and *run* are not grouped. Evaluation words were removed from all training files by family before training.

The released adapter was trained on 4,764 pairs: screened web pairs, recovered pairs and ALT-attested spellings. In one earlier experiment we also added 384 spellings suggested by Google's AI Mode; they made no measurable difference and are not used by either released model.

3 Measuring acceptable transliteration

Our first metric was the usual one: how often the model's first answer exactly matched a reference spelling, with references taken from the mined data. For a task with several acceptable answers and noisy references, this metric fails in two directions.

It can reject an acceptable spelling. For the development word *band*, the released adapter wrote បេន, which a Khmer reader pronounces close to the English word. The mined reference was បាន, so the metric scored the

adapter as wrong. It can also reward an unacceptable one. Earlier models wrote បាន for *band*; it matched the reference and scored as correct, yet បាន is also an everyday Khmer word read roughly "baan", and the reviewer judged it not acceptable. The same pattern appeared for *phi*: the released adapter's ភី matched the reference and was rejected, while an earlier model's ហ្វី missed it and was accepted.

These were not isolated cases. In the development review described below, 28 of the 72 spellings that differed from the mined reference were acceptable, and 8 of the 37 that matched it were not. In a separate diagnostic on hard words, the reviewer accepted two or three different candidate spellings for 12 of 23 words. Exact agreement with one reference therefore measures resemblance to a noisy label, not phonetic acceptability. Shared tasks with reference *lists* [4] address the first failure but not the second.

Blinded review. We replaced single-reference scoring with human judgments under a written policy fixed before any review. For each word the reviewer saw the English word and its intended reading in the dictionary's pronunciation symbols (for example `B AE1 N D` for *band*). Below it were the candidate spellings from the compared models, with model identities, ranks and scores hidden, in random order and with duplicates shown once. Each spelling received one of three labels: *acceptable* if a Khmer reader would pronounce it close to the intended reading, *not acceptable*, or *uncertain*. The reviewer could accept several spellings or none and never had to type a spelling. "Acceptable" in our results means confirmed acceptable; uncertain answers stay uncertain and count as not confirmed.

Development review and prospective tests. Two kinds of review must be kept apart. The *development review* looked back at existing models on 60 development words, the set also used to select checkpoints: 40 sampled to be representative and 20 chosen because the models disagreed. Its answers were given in free text and mapped to the three labels under rules the reviewer confirmed, for example that a candidate not named in the reply is not acceptable. It guided decisions but is exploratory. The two *prospective tests* each used 120 fresh words (40 ordinary words, 40 irregularly spelled words and 40 names), chosen and frozen before the candidate model was trained, with all outputs frozen before review. Each spelling was judged with an explicit click on a review page. The two test samples are different, so their rates are not comparable with each other.

All judgments come from one reviewer, the project's author, who is a Khmer reader but not independent of the project. The results are therefore task-specific evidence for one reader and one policy. The 40/40/40 design deliberately over-represents hard words and names, so the rates do not estimate performance on everyday text.

4 What changed model quality?

Table 2 summarises what we observed and the main limitation of each finding; the sections below explain them. Reference agreement for the adapted models is measured on a fixed *historical* evaluation set of 228 words that was examined across many experiments, so those comparisons are exploratory.

4.1 More real data: encouraging, if measured well

Why we tried it. The filtering mistake and the consonant check had discarded many genuine pairs. Recovering them and adding corpus-attested spellings roughly doubled the adapter's data.

What happened. Reference agreement barely moved: the larger-data adapter and the first working adapter matched the mined references about equally often. The development review told a different story. The larger-data model's spelling was acceptable more often in both the representative and the disagreement-enriched samples, which are reported separately because they were drawn differently. A version that also used the AI-suggested spellings performed like the real-data version, so the gain did not depend on them.

What it means. The data change looked useful only once we measured acceptability, so the strength of this conclusion rests on the evaluation. Because the review was post hoc and had one reviewer, we treat it as an encouraging development result rather than an established improvement, and we released the real-data model.

4.2 Longer training: no demonstrated benefit

Why we tried it. The selected checkpoint of the released adapter was the last one evaluated, which raised the possibility that it was under-trained.

What happened. Doubling the number of training updates changed 65 of the 120 test spellings but did not change the overall acceptance rate: both models were acceptable for 77 of 120 words, each winning on 14 words the other lost. The sample was too small to rule out modest differences in either direction.

What it means. More training rearranged the errors without reducing them. With noisy, incomplete supervision, training longer is not a reliable fix; the model may simply fit the noise differently.

4.3 Pronunciation alongside letters: no clear benefit

Why we tried it. English spelling is a poor guide to English sound, and earlier transliteration work gained from combining letters with pronunciation [10, 11]. Supplying the dictionary pronunciation seemed likely to help with words like *colonel*.

What happened. For the small model, we gave the pronunciation symbols as a second input next to the

Table 2: What we observed. "No clear benefit" means the samples could not detect a difference; it does not mean the methods are equivalent. Paired differences and intervals are in Appendix A, Table 3.

Question	Observed outcome	Main limitation
More real data	Encouraging: the development review preferred the larger-data adapter, while agreement with mined references barely moved.	Exploratory, post hoc review of development words by one reviewer; not a confirmed improvement.
Longer training	Equal observed acceptance on test sample 1 (77 of 120 for both models), although 65 spellings changed. No clear benefit detected.	One training run and 120 words; modest differences in either direction cannot be ruled out.
Pronunciation input	No clear benefit detected, for the small model or for the pronunciation-guided adapter (90 of 120 for both on test sample 2).	The adapter experiment changed several ingredients together, so the effect of pronunciation is not isolated.
Decoding	Disabling early stopping removed the apparent beam-search deficit.	Measured by reference agreement; beam search then showed no clear advantage over greedy decoding.

letters. Across three comparisons on different data the differences were small and uncertain; the registered comparison is in Appendix A. A diagnostic explains part of this. When we swapped in another word's pronunciation, the model's first answer changed for 33.0–44.7% of words, depending on the training run, but the changed answers rarely moved towards the other word's consonants; most changes were vowel or diacritic edits.

For the adapted model, we appended the pronunciation to the input. A *word-only* input such as `fire` became the *pronunciation-guided* input `fire|F AY1 ER0`, where the symbols spell the English sounds and the digits mark stress (1 for primary stress, 0 for none). No audio is involved, and the output is still Khmer text. This model also received the reviewer's accepted spellings for 42 difficult training word families and started from the released adapter. On the second test sample it was acceptable for exactly as many words as the released adapter. Removing the pronunciation changed only about 7.1% of its development outputs, so the model still relied mainly on the letters.

What it means. In these models and data, extra pronunciation input was largely ignored. This does not show that pronunciation cannot help Khmer transliteration, and because the second test changed several ingredients together, it cannot isolate the effect of pronunciation. A model may need training data in which spelling and sound disagree often enough to force it to read the pronunciation.

4.4 A decoding setting, not the model, caused a beam-search deficit

Why we looked. For the adapted models, beam search, which keeps several partial spellings and returns the most probable complete one, scored worse than simply taking the most likely byte at each step (greedy decoding). That is unusual.

What happened. The beam search had been called with early stopping enabled. At the byte level, junk hypotheses that end after one or two bytes finish first and fill the set of completed answers before real spellings end. The search then returned truncated

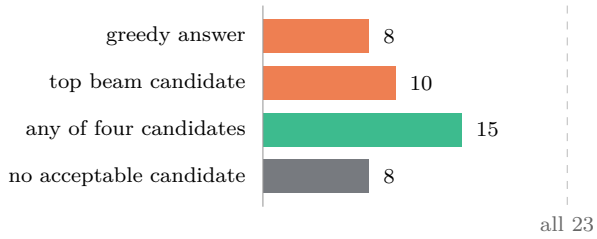


Figure 3: Candidate coverage on 23 hard development words: the number of words whose greedy answer, top beam candidate or any of four beam candidates was acceptable, and the number with no acceptable candidate. The gap between the first and third bars is what a perfect selector could gain; it is a best-case limit, not the accuracy of an implemented selector.

or less probable spellings: for the development word *mani*, greedy decoding gave ម៉ានី but the beam returned មានី, dropping the mark that sets the consonant's series. With early stopping disabled, beam search matched greedy decoding and clearly beat the original beam setting (Table 3 in Appendix A).

What it means. Check generation settings before attributing a difference to model quality. Earlier beam-search comparisons in this project were shifted by the defect, so we report greedy results for the adapted models.

4.5 Acceptable spellings are often generated but not chosen

Why we looked. Irregularly spelled words were the weakest group in both tests: the released adapter was acceptable for 14 and 20 of 40 such words. We wanted to know whether the model could not produce an acceptable spelling at all, or produced one without ranking it first.

What happened. For 23 irregular development words, the reviewer judged every distinct spelling among the model's greedy answer and its four beam-search candidates (Figure 3). The greedy answer was acceptable for 8 words and the top beam candidate for 10, but at least one of the four candidates was acceptable for 15. For 8 words no candidate was acceptable.

What it means. There are two separate problems:

selecting an acceptable candidate that already exists, and *generating* one when none does. A perfect selector over four candidates would have added 7 words here, which suggests that rescoring candidates [5] is worth trying. But this is a best-case limit on 23 hand-picked hard words, not the performance of a selector we built, and it cannot tell us whether selection or generation matters more on ordinary text.

5 Practical lessons and limitations

Several lessons should transfer to other low-resource transliteration projects.

- *Audit what filtering rules remove.* Our miner silently discarded every pair whose Khmer side was a dictionary word. Looking at examples of dropped data, not just counts, would have caught it at once.
- *Check the generation configuration before blaming the model.* A single decoding flag produced an apparent beam-search deficit that looked like a modelling problem.
- *Do not confuse agreement with one reference with acceptability.* When several spellings are valid and references are noisy, exact matching rejects good outputs and rewards bad references. Blinded judgments with explicit "uncertain" answers were more informative than any automatic score we had.
- *More training does not repair conflicting or incomplete supervision.* A longer schedule changed many outputs without improving them.
- *Treat candidate coverage as an opportunity, not a result.* Acceptable alternatives in a beam show that a selector could help; they are not the accuracy of one.

Limitations. Irregular English spellings remain the main weakness: on the two tests, the released adapter was acceptable for 35–50% of irregular words. The training data are mined and unreviewed, and the automatic checks do not cover vowels. All human judgments come from one project-affiliated reviewer on samples of 120 words, so differences of several percentage points cannot be resolved, and each adapter was trained once. Much of the reference-based evidence uses one historical evaluation set that was examined across many experiments. The two released models were trained on different data and were never evaluated on a common, untouched test, so we do not compare them with each other. Only single words are handled; sentence context is out of scope.

Availability. Code and documentation: `https://github.com/richardcheam/khmer-transliterator`. Small model: `https://huggingface.co/ric912/en-km-phonetic-transliterator` [3]. Adapter: `https://huggingface.co/ric912/en-km-phonetic-byt5-lora` [2]. Both models are released under CC BY-NC-SA 4.0. Training pairs come from FineWeb-2 (ODC-By 1.0, with Common Crawl's terms) [7], Khmer Wikipedia [17] and the ALT Parallel Corpus (CC BY 4.0) [9]; rights in the underlying web text remain with its authors, and the repository's notices give details. Training pairs, judgments and references are not distributed.

6 Conclusion

Spelling English words in Khmer script is a small task with an awkward property: many answers are right, and we found no reliable list of them. With only noisy web pairs for supervision, the most consequential change in this project was not a model but a measurement: replacing exact agreement with one reference by blinded judgments of acceptability. Under that measurement, cleaner and larger real data looked encouraging, while longer training and added pronunciation information showed no clear benefit. The released adapter produces an acceptable spelling for 64.2% and 75.0% of words in two separate, deliberately hard 120-word samples, and for about half or fewer of the irregularly spelled ones. Acceptable alternatives are often among its candidates, which points to candidate selection and reviewed data as the next steps.

References

[1] Carnegie Mellon University Speech Group. CMUdict: The Carnegie Mellon pronouncing dictionary. `https://github.com/cmusphinx/cmudict`. Accessed 2026-10-31.

[2] Richard Cheam. en-km-phonetic-byt5-lora (model repository). `https://huggingface.co/ric912/en-km-phonetic-byt5-lora`, 2026. Adapter weights CC BY-NC-SA 4.0; revision 77f36282.

[3] Richard Cheam. en-km-phonetic-transliterator (model repository, tag v0.2.0). `https://huggingface.co/ric912/en-km-phonetic-transliterator`, 2026. Weights CC BY-NC-SA 4.0.

[4] Nancy Chen, Xiangyu Duan, Min Zhang, Rafael E. Banchs, and Haizhou Li. NEWS 2018 whitepaper. In *Proceedings of the Seventh Named Entities Workshop*, pages 47–54, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-2408. URL `https://aclanthology.org/W18-2408/`.

[5] Roman Grundkiewicz and Kenneth Heafield. Neural machine translation techniques for named entity transliteration. In *Proceedings of the Seventh Named Entities Workshop*, pages 89–94, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-2413. URL `https://aclanthology.org/W18-2413/`.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2106.09685.

[7] Hugging Face. FineWeb-2 dataset card. https://huggingface.co/datasets/HuggingFaceFW/fineweb-2. Revision af9c1333; ODC-By 1.0, use also subject to Common Crawl's Terms of Use. Accessed 2026-10-31.

[8] Christo Kirov, Cibu Johny, Anna Katanova, Alexander Gutkin, and Brian Roark. Context-aware transliteration of Romanized South Asian languages. *Computational Linguistics*, 50(2):475–534, 2024. doi: 10.1162/coli_a_00510. URL https://aclanthology.org/2024.cl-2.2/.

[9] National Institute of Information and Communications Technology (NICT). ALT parallel corpus. https://www2.nict.go.jp/astrec-att/member/mutiyama/ALT/. Release 2019-12-06; licensed CC BY 4.0. Accessed 2026-10-31.

[10] Hoang Gia Ngo, Nancy F. Chen, Binh Minh Nguyen, Bin Ma, and Haizhou Li. Phonology-augmented statistical transliteration for low-resource languages. In *Proc. Interspeech 2015*, pages 3670–3674, 2015. doi: 10.21437/Interspeech.2015-728. URL https://www.isca-archive.org/interspeech_2015/ngo15_interspeech.html.

[11] Jong-Hoon Oh and Key-Sun Choi. An ensemble of grapheme and phoneme for machine transliteration. In *Natural Language Processing – IJCNLP 2005 (Second International Joint Conference on Natural Language Processing)*, volume 3651 of *Lecture Notes in Computer Science*, pages 450–461. Springer, 2005. doi: 10.1007/11562214_40. URL https://aclanthology.org/I05-1040/.

[12] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. FineWeb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025. URL https://arxiv.org/abs/2506.20920.

[13] Hammam Riza, Michael Purwoadi, Gunarso, Teduh Uliniansyah, Aw Ai Ti, Sharifah Mahani Aljunied, Luong Chi Mai, Vu Tat Thang, Nguyen Phuong Thai, Vichet Chea, Rapid Sun, Sethserey Sam, Sopheap Seng, Khin Mar Soe, Khin Thandar Nwet, Masao Utiyama, and Chenchen Ding. Introduction of the Asian Language Treebank. In *2016 Conference of the Oriental Chapter of International Committee for Coordination and Standardization of Speech Databases and Assessment Techniques (O-COCOSDA)*, pages 1–6, 2016. doi: 10.1109/ICSDA.2016.7918974.

[14] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27, 2014. URL https://arxiv.org/abs/1409.3215.

[15] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016. URL https://arxiv.org/abs/1512.00567.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

[17] Wikimedia Foundation. Terms of use. https://foundation.wikimedia.org/wiki/Policy:Terms_of_Use. Section 7: text under CC BY-SA 4.0 and GFDL. Accessed 2026-10-31.

[18] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10: 291–306, 2022. doi: 10.1162/tacl_a_00461. URL https://aclanthology.org/2022.tacl-1.17/.

A Methods in brief

Models and training. *Small model:* encoder–decoder Transformer, model width 192, three encoder and three decoder layers, four attention heads, feed-forward width 512; AdamW, learning rate 10^{-3} with cosine decay, label smoothing 0.1, 40 epochs with the epoch chosen on 86 development items; beam search of width 4. The input is a task marker followed by the letters; the output vocabulary holds 78 Khmer Unicode characters plus start and end symbols. *Adapted model:* ByT5-small (width 1472, 12 encoder and 4 decoder layers, gated feed-forward blocks), all of whose weights stay frozen. LoRA matrices of rank 16 with $\alpha = 32$ are added to the query, key, value and output projections of every attention block (encoder self-attention, decoder self- and cross-attention) and to the three projections of every feed-forward block (6,258,688 trained parameters); the embeddings, output layer and layer norms are not adapted. Inputs are UTF-8 bytes plus an end symbol. Training used AdamW, learning rate 10^{-3}, 100 warm-up steps then linear decay, batch 32, 3,000 updates on a laptop CPU, one training run; checkpoint chosen by exact agreement on 253 development items, then character error rate. Single answers use greedy decoding; candidate lists use beam search with early stopping disabled. Full configurations are in the repository's experiment folders and model cards.

Training objectives. Both models are trained with teacher forcing: the decoder input is the start symbol followed by the reference $y_1, \ldots, y_{T-1}$, and the targets are $y_1, \ldots, y_T$, ending with the end symbol. Batches are right-padded; padded source positions are masked in encoder self-attention and in cross-attention, and padded target positions are excluded from the loss. For a batch with N real target tokens and per-token losses $\ell_{n,t}$, the loss is $\frac{1}{N}\sum_n \sum_{t \le T_n} \ell_{n,t}$, a mean over tokens, not over words. Each recorded (word, spelling) pair is one example: 2,264 of the small model's 6,717 training words and 840 of the adapter's 3,585 have more than one recorded spelling, and their pairs are trained independently. For the adapted model $\ell_{n,t} = -\log p_\theta(y_{n,t} \mid y_{n,<t}, x_n)$, as in Equation (2). For the small model, with smoothing $\varepsilon = 0.1$ spread uniformly over all $K = 81$ output classes (78 Khmer characters plus the padding, start and end symbols),

$$\ell_{n,t} = -(1-\varepsilon)\log p_\theta(y_{n,t} \mid y_{n,<t}, x_n) - \frac{\varepsilon}{K}\sum_{c=1}^{K} \log p_\theta(c \mid y_{n,<t}, x_n). \tag{5}$$

LoRA details. In Equation (3), a projection computes $h = W_0 z + \frac{\alpha}{r} BAz$ for its input z. ByT5-small has 6 attention heads of size 64, so its attention projections map between the model width 1472 and an inner width of 384: $W_0 \in \mathbb{R}^{384\times1472}$ for the query, key and value projections and $\mathbb{R}^{1472\times384}$ for the output projection. Its gated feed-forward blocks have two input projections $W_0 \in \mathbb{R}^{3584\times1472}$ and one output projection $W_0 \in \mathbb{R}^{1472\times3584}$. Adapters sit on all 128 such projections (four in each of 20 attention blocks: 12 encoder self-attention, 4 decoder self-attention, 4 cross-attention; three in each of 16 feed-forward blocks), giving 256 matrices and 6,258,688 parameters. A is initialised with Kaiming-uniform values and B with zeros; during training, dropout of 0.05 is applied to z on the adapter path only, and the frozen model keeps its dropout of 0.1. At inference there is no dropout.

Data. The adapter's 4,764 pairs (3,585 words) are 2,255 screened web pairs, 1,292 pairs recovered from the consonant check and 1,217 ALT-attested pairs. The small model used 9,735 web pairs mined before the filtering repair.

Table 3: Detailed comparisons behind Table 2. Each row compares two systems on the same words. Acceptability counts are confirmed-acceptable spellings out of the words reviewed; reference agreement is the share of first answers that exactly match the mined reference. Differences are in percentage points with a 95% paired family-bootstrap interval; an interval that includes zero means no clear difference, not equivalence. The two test samples contain different words.

Question	Comparison	Evidence	Result	Difference [95%]
More real data	released adapter vs first working adapter	reference agreement, 228 historical evaluation words	52.3% vs 49.9%	+2.5 [−2.7, +7.7]
	same models	development review, 40 representative words	29 vs 24 acceptable	exploratory
	same models	development review, 20 words where models disagreed	10 vs 5 acceptable	exploratory
Longer training	twice the training updates vs released adapter	test sample 1, 120 fresh words	77 vs 77 acceptable	0.0 [−8.3, +8.3]
Pronunciation	small model, letters + pronunciation vs letters only	curated references, 162 word families, three paired training runs		+1.6 [−1.6, +4.9]
	pronunciation-guided adapter vs released adapter	test sample 2, 120 fresh words	90 vs 90 acceptable	0.0 [−7.5, +7.5]
Decoding	corrected beam search vs original beam setting	reference agreement, 228 historical evaluation words; one earlier adapter		+5.9 [+1.4, +10.4]
	corrected beam search vs greedy decoding	same		+0.9 [−1.4, +3.6]

Evaluation and statistics. Paired differences use a bootstrap over word families with 10,000 resamples; the intervals are conditional on the trained checkpoints and do not include variation between training runs. The small model's curated references cover 165 items in 162 families; for 82 of them, and 94 of 191 items across both final reference tracks, the reference was chosen from an earlier model's candidates. The historical evaluation set overlaps the small model's training words: 110 of its 228 primary items (122 of 245 words in the full partition), which is one reason the two released models are not compared. The development review's free-text answers were mapped to labels under three rules the reviewer confirmed; the prospective tests and the candidate-coverage judgments were clicks on a review page (3 of the 93 coverage judgments were reused from the development review for the identical word, reading and spelling). Further comparisons behind Section 4.3: on consonant-screened pairs, letters + pronunciation vs letters +0.6 [−2.4, +3.8]; in a second three-run comparison +2.2 [−1.5, +6.0]; pooling candidates from both models vs a letters model's own eight candidates +1.6 [−1.1, +4.5]; the pronunciation-guided adapter on the historical evaluation set −1.3 [−4.8, +2.3]. Sources for every number are listed in the report's evidence ledger.

Training curves. Figure 4 shows how both released models trained, from their logged training records. The small model's held-out loss is lowest at epoch 14 (1.217) and then rises slightly, to 1.234 at epoch 40, while the training loss keeps falling: mild overfitting in the loss. Its development score rises to about 20% and then moves by a word or two (one of the 86 items is 1.2 points); the selected epoch, 27, is one of two epochs tied at the top, chosen by the lower held-out loss. The adapter's loss falls within the first few hundred updates; its development score rises most between 600 and 1,200 updates and slowly after that, up to the last update, which was selected. Gradient norms, learning rate and memory were also logged; they are not shown.

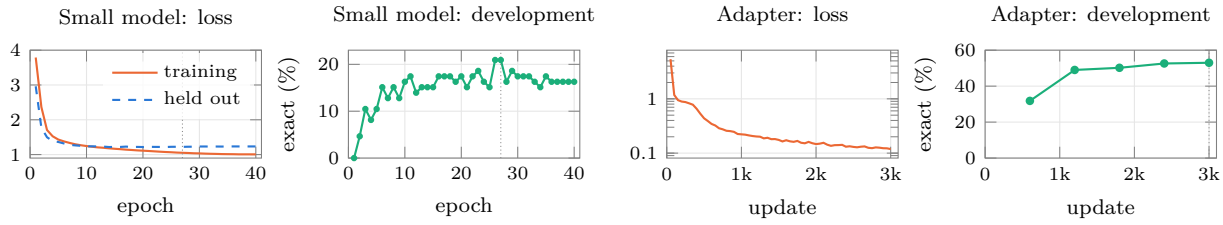


Figure 4: Training curves of the released models, from their training logs (adapter loss on a log scale). Small model: training loss per epoch (label smoothing 0.1), loss on pairs held out from the training file, and greedy exact agreement on the 86 development items. Adapter: training loss averaged over 50-update blocks, and greedy exact agreement averaged over word families every 600 updates. Dotted lines: selected checkpoints. The development sets are small and scored against recorded spellings, not the acceptability review.

B Worked examples

All examples are recorded outputs, judgments or training pairs from the project; none comes from a protected reference set. "Reference" is the mined spelling used for exact-agreement scoring. Development words were excluded from training but belong to the set used to select checkpoints; they are not test words.

English word (role)	Intended reading	Input and system	Output	Reference / judgment	Lesson
facebook (training pair)	`F EY1 S B UH2 K`	bracket match in web text	ហ្វេសប៊ុក	n/a	A genuine loan that the original filter discarded because its Khmer side is a dictionary word.
coffee (training pairs)	`K AA1 F IY0`	three mined spellings	កាហ្វេ, កូហ្វី, ខូហ្វី	n/a	Web data mixes the established loan with pronunciation-based spellings.
hotel (training pair)	`HH OW0 T EH1 L`	pair recovered after the consonant check	តែល	n/a	Recovered pairs are unreviewed and can be truncated.
band (development)	`B AE1 N D`	`band`; released adapter	បេន	ref. បាន; acceptable	Exact matching rejects an acceptable spelling.
band (development)	`B AE1 N D`	`band`; earlier adapters	បាន	matches ref.; not acceptable	Exact matching rewards a poor reference.
phi (development)	`F AY1`	`phi`; released vs first working adapter	ភី vs ហ្វី	ភី matches ref. but not acceptable; ហ្វី acceptable	The same failure in the opposite direction for the released model.
cameroon (development)	`K AE1 M ER0 UW2 N`	`cameroon`; released adapter	ខេមឺរ៉ុន	differs from ref.; acceptable	Vowel choices often differ from the reference without being wrong.
mani (development)	`M AA1 N IY0`	`mani`; greedy vs original beam setting	ម៉ានី vs មានី	n/a	The early-stopping beam dropped a series mark.
fire (input format)	`F AY1 ER0`	word-only `fire`; pronunciation-guided `fire|F AY1 ER0`	n/a	n/a	The pronunciation is text appended to the input; no audio is used.
(evaluation words)	n/a	first adaptation, attention layers only	ហ្វេន for 204 of 228 words	n/a	A falling byte-level loss can hide a model that ignores its input.